Trabalho Prático — Lidando com sensores em dispositivos móveis

Disciplina: DGT2816 — Interação com sensores de smartphones e wearables **App:** Doma Assist (`com.doma.assist`) — Wear OS, Kotlin

1. Contextualização

A empresa "Doma" quer um aplicativo Wear OS para assistência a funcionários com necessidades especiais, usando **áudio** para fornecer informações em tempo real: leitura de mensagens de texto, notificações, lembretes e respostas a comandos de voz — útil especialmente para pessoas com deficiência visual —, além de alertas de segurança sonoros (emergências, avisos críticos).

2. Requisitos atendidos (mapeamento com o roteiro)

Item do roteiro	Implementação
Configuração do ambiente	Projeto Android (Kotlin), `minSdk 30` , criado para Wear OS
Leitura de mensagens/notificações em voz alta	Mensagens chegam automaticamente → notificação real + o app pergunta por voz "*Nova mensagem. Deseja ouvir?*" → usuário responde "sim"/"não" falando
Comandos de voz acessíveis	Escuta **sob demanda**: toque duplo na tela, gesto de agitar (acelerômetro) ou botão físico (`KEYCODE_STEM_1-3`); mic pausa durante a fala do TTS
Saídas de áudio: `TYPE_BUILTIN_SPEAKER`	`AudioHelper.audioOutputAvailable(AudioDeviceInfo.TYPE_BUILTIN_SPEAKER)`
Saídas de áudio: `TYPE_BLUETOOTH_A2DP`	`AudioHelper.audioOutputAvailable(AudioDeviceInfo.TYPE_BLUETOOTH_A2DP)`
`getDevices()` + `FEATURE_AUDIO_OUTPUT`	`AudioManager.getDevices(GET_DEVICES_OUTPUTS)` guardado por `hasSystemFeature(FEATURE_AUDIO_OUTPUT)`
Detecção dinâmica via `registerAudioDeviceCallback`	Callback registrado em `onResume` / `onPause` ; atualiza status e registra mensagem quando o fone Bluetooth conecta/desconecta

Item do roteiro	Implementação
Intent `ACTION_BLUETOOTH_SETTINGS`	Comando de voz "*bluetooth*" abre as configurações com `EXTRA_CONNECTION_ONLY`, `EXTRA_CLOSE_ON_CONNECT` e `FILTER_TYPE = 1`
Reprodução de áudio	`TextToSpeech` (leitura de mensagens/alertas) e `ToneGenerator` (tom de alerta)
Alto-falantes em Wear OS	Saída exibida na tela; TTS usa a saída disponível (alto-falante ou fone)
Notificações de emergência	Comando "*alerta*" posta notificação real (canal `alertas`, prioridade alta, com vibração) + tom + TTS

3. Arquitetura

```
com.doma.assist
├── MainActivity.kt              Tela (sem botões — interface auditiva): lista
│                                de mensagens, status de áudio, TTS, fluxo de
│                                pergunta/resposta e fila de mensagens
├── AudioHelper.kt               Detecção de saídas de áudio (AudioManager) e
│                                registro do AudioDeviceCallback
├── VoiceCommandHelper.kt        SpeechRecognizer em sessões sob demanda
│                                (mic só liga quando necessário)
├── DomaNotificationListener.kt  NotificationListenerService que falaria
│                                notificações em voz alta (ver §4.1)
├── DomaTileService.kt           Tile de tela cheia no mostrador: um deslize +
│                                toque em qualquer lugar abre o app
└── DomaComplicationService.kt   Complication "Doma" na watch face: atalho com
                                 descrição falada pelo TalkBack
```

Decisão de acessibilidade

O app foi pensado para usuários **cegos**: a interface visual é mínima (sem botões para "achar na tela") e toda a interação é **auditiva + gestos simples**. O microfone nunca fica ligado continuamente — ele só ativa quando o app pergunta algo ou quando o usuário aciona (toque duplo, agitar o pulso ou botão físico). Mensagens que chegam durante uma interação ou com o app fora da tela entram em fila e são anunciadas em sequência, sem sobrepor avisos.

Fluxo principal

1. Ao abrir, o app enumera as saídas de áudio e mostra o status na tela.

2. **Chegada automática**: a cada ~30s uma nova mensagem/alerta é recebida, adicionada à lista e postada como notificação (`NotificationCompat` , canal `alertas` , prioridade alta). O app então pergunta por voz "*Nova mensagem. Deseja ouvir?*" e aguarda a resposta falada — **"sim"** lê a mensagem, **"não"** ou silêncio a ignora.
3. **Comandos por voz**: toque duplo na tela (ou agitar o relógio / botão físico) → o app fala "*Ouvindo…*" → o usuário fala o comando. O microfone só fica ativo nesses momentos e pausa enquanto o app fala.
4. **Alerta**: emite um tom de alarme e fala a mensagem de segurança.
5. **Bluetooth**: abre as configurações de conexão do relógio.
6. O `AudioDeviceCallback` mantém o status atualizado em tempo real.

Reabrir o app a partir do mostrador

- **Tile**: deslize no mostrador → cartão de tela cheia "*Doma Assist*" → toque em qualquer lugar abre o app (adicionar uma vez: toque longo na tile → "+" → Doma Assist)
- **Complication**: ícone "Doma" na watch face (adicionar uma vez editando o mostrador); com TalkBack a descrição "Abrir Doma Assist" é falada
- **Notificação**: chega com vibração → tocar nela abre o app
- **Voz**: "*Ok Google, abrir Doma Assist*" (dispositivos com Assistant)
- Mensagens que chegam fora da tela ficam na fila e são anunciadas ao reabrir o app

4. Permissões (AndroidManifest.xml)

- `RECORD_AUDIO` — reconhecimento de voz (solicitada em runtime)
- `POST_NOTIFICATIONS` — postar notificações de alerta (solicitada em runtime)
- `BLUETOOTH` / `BLUETOOTH_CONNECT` — pareamento de fones
- `MODIFY_AUDIO_SETTINGS` — roteamento de áudio
- `BODY_SENSORS` , `WAKE_LOCK` — exigidas pela prática
- `uses-feature android.hardware.type.watch` — restringe a wearables

4.1 Nota técnica — leitura de notificações de terceiros

O app inclui `DomaNotificationListener` , um `NotificationListenerService` implementado para falar o título e o texto de cada notificação recebida. Contudo, o **Wear OS 3+ não concede acesso a notificações de outros apps a aplicativos de terceiros** — não existe a tela "Acesso a notificações" no relógio e o sistema rejeita o serviço. A leitura em voz alta, portanto, é feita sobre as mensagens e alertas do próprio app (fluxo de confirmação por voz e comando "*ler mensagens*"), atendendo ao mesmo objetivo de acessibilidade.

5. Como testar

1. Abrir `Doma/` no Android Studio e aguardar o Gradle Sync.
2. Criar/selecionar emulador Wear OS (Small Round, API 30+) ou usar um smartwatch físico.
3. Executar o app. Na primeira interação de voz, permitir o acesso ao microfone. No emulador: *Extended Controls → Microphone → Enable Host Microphone Access* para reconhecer voz do PC.
4. Aguardar ~30s: o app pergunta "*Nova mensagem. Deseja ouvir?*" → responder **"sim"** (lê) ou **"não"**.
5. Toque duplo na tela → "*Ouvindo...*" → falar "*que horas são*", "*alerta*", "*ler mensagens*" ou "*bluetooth*".
6. Adicionar a tile no mostrador (opcional): deslizar até o carrossel de tiles → toque longo → "+" → *Doma Assist* → depois basta um deslize e um toque para reabrir o app.

6. Capturas de tela

Inserir as capturas em `docs/capturas/`:

- [ ] Tela principal no emulador Wear OS
- [ ] Leitura de mensagens / alerta em execução
- [ ] Captura via app complementar do Wear (smartphone)

7. Resultados

O app demonstra os conceitos do roteiro: detecção de saídas de áudio, monitoramento dinâmico de dispositivos, facilitação de conexão Bluetooth e reprodução de áudio (TTS + tons), aplicados a um caso de acessibilidade no trabalho.